

category: dotnet-general

tags: [git, ref-corruption, reflog, recovery, windows, update-ref, fsck]

severity: high

created: 2026-07-26

updated: 2026-07-26

project-origin: App-Access-V2 (migrate WinForms → Avalonia, STEP-0.3)

`.git/refs/heads/<branch>` hỏng thành toàn byte NUL — "branch appears to be broken", khôi phục từ reflog KHÔNG mất commit

Tình huống gặp phải

Ngay sau khi một bước plan commit xong (`git add + git commit`, không push), session tiếp theo mở lại repo thì **mọi lệnh git đều fail**. Máy Windows 11, repo trên ổ E:, nhánh `dev`. Commit vừa tạo có tên trong step file nhưng không cách nào xác minh được.

Triệu chứng / Lỗi

```
$ git status
fatal: your current branch appears to be broken

$ git rev-parse HEAD
fatal: ambiguous argument 'HEAD': unknown revision or path not in the working tree.

$ git update-ref refs/heads/dev <sha>
fatal: update_ref failed for ref 'refs/heads/dev': cannot lock ref 'refs/heads/dev':
unable to resolve reference 'refs/heads/dev': reference broken
```

`.git/HEAD` vẫn đúng (`ref: refs/heads/dev`), file `.git/refs/heads/dev` vẫn tồn tại và **đúng 41 byte** (bằng độ dài SHA-1 + newline) — nhưng nội dung là **toàn byte NUL**:

```
$ cat .git/refs/heads/dev
                                <-- rỗng khi in ra

$ git cat-file -t $(cat .git/refs/heads/dev)
bash: warning: command substitution: ignored null byte in input
```

Nhánh `main` bên cạnh vẫn bình thường → chỉ nhánh đang active bị hỏng.

Nguyên nhân gốc rễ (Root Cause)

Git ghi ref bằng cách tạo file `<ref>.lock`, ghi SHA, rồi `rename()` đè lên file ref. Trên Windows/NTFS, nếu tiến trình bị kill hoặc máy mất điện **giữa lúc filesystem đã cấp phát 41 byte nhưng chưa flush nội dung**, file ref còn lại đúng kích thước nhưng dữ liệu là NUL — metadata đã commit, data chưa. Đây là ref corruption cổ điển sau crash, không phải hỏng repository.

Điểm mấu chốt: commit object nằm trong `.git/objects/`, hoàn toàn độc lập với file ref. Ref chỉ là con trỏ 41 byte. Ref hỏng ⇒ mất con trỏ, **KHÔNG** mất commit.

Giải pháp

Reflog (`.git/logs/refs/heads/<branch>`) là file text append-only, không đi qua cơ chế lock/rename nên thường còn nguyên — dùng nó để lấy lại SHA.

```
# 1. Lấy SHA cuối cùng từ reflog dạng file (KHÔNG dùng `git reflog` - lệnh này cũng cần resolve ref)
tail -1 .git/logs/refs/heads/dev
# <old-sha> <NEW-SHA> Author <mail> 1785054161 +0700 commit: <message>
#          ^^^^^^^ cột 2 là SHA đích cần khôi phục

# 2. Xác minh commit object còn sống TRƯỚC khi động vào ref
git cat-file -t 98428010caca9a64cb9a0f27120a66d1cc7f2761 # -> commit
git cat-file -p 98428010caca9a64cb9a0f27120a66d1cc7f2761 # xem tree/parent/message

# 3. BẮT BUỘC xoá file ref hỏng trước - update-ref không lock được file hỏng
rm -f .git/refs/heads/dev .git/refs/heads/dev.lock

# 4. Tạo lại ref
git update-ref refs/heads/dev 98428010caca9a64cb9a0f27120a66d1cc7f2761

# 5. Verify
git log --oneline -5
git fsck --no-progress --connectivity-only
```

Sau bước 5: working tree, index, staged changes đều còn nguyên — không cần `reset`, không cần clone lại.

Áp dụng lại (How to reuse)

- Thấy `fatal: your current branch appears to be broken` → **KHÔNG** clone lại, **KHÔNG** `git reset --hard`, **KHÔNG** xoá `.git`. Kiểm tra ngay `ls -la .git/refs/heads/` — file 41 byte mà `cat` ra rỗng = NUL corruption.
- Luôn lấy SHA từ file `.git/logs/refs/heads/<branch>` (hoặc `.git/logs/HEAD`), không dùng lệnh `git reflog` vì lệnh này cũng phải resolve ref đang hỏng.
- Nếu reflog cũng mất → `git fsck --lost-found` rồi soi các `dangling commit` để tìm lại.
- Sau khi khôi phục luôn chạy `git fsck --connectivity-only` để chắc chắn object store không thiếu gì.

Chú ý / Cạm bẫy (Gotchas)

- ⚠ `git update-ref` **thất bại** nếu file ref hỏng còn nằm đó (`cannot lock ref ... reference broken`) — phải `rm` file ref trước, đây là bước hay bị bỏ sót và khiến người ta tưởng repo hỏng thật.
- ⚠ Đừng thử `git checkout <branch>` / `git switch` để "sửa" — cũng phải resolve ref hỏng nên fail tiếp, và có thể tạo thêm `.lock`.
- ⚠ `$(cat .git/refs/heads/<branch>)` trong bash sẽ nuốt byte NUL và trả chuỗi rỗng ⇒ lệnh dùng nó báo lỗi sai lệch ("`<object> required`") làm hiểu nhầm là sai cú pháp.
- ⚠ Ref hỏng chỉ ảnh hưởng nhánh đó — các nhánh khác và `packed-refs` (chứa remote refs) vẫn dùng được, có thể đối chiếu `origin/<branch>` để biết repo còn lành.

- ⚠ Nếu commit vừa tạo **chưa push**, đây là bản duy nhất — xác minh `git cat-file -t` trước khi làm bất kỳ thao tác ghi nào lên `.git/`.

Tham chiếu

- Project: `App-Access-V2` — nhánh `dev`, commit `9842801` (STEP-0.3 plan migrate Avalonia) khôi phục thành công, không mất dữ liệu.
- Ghi chú sự cố trong repo: `docs/plans/PLAN-migrate-avalonia-2026-07-26/steps/STEP-0.3-linux-compat-lib.md` mục "Sự cố hạ tầng git đã xử lý".